



AWS ECS (Elastic Container Service)



Table of Contents

1. Overview of ECS
2. Core ECS Components
3. ECS Launch Types: EC2 vs Fargate
4. ECS Networking
5. ECS Security Best Practices
6. Monitoring and Logging
7. CI/CD Pipeline with ECS
8. Scaling Strategies
9. Deployment Strategies
10. Cost Optimization
11. Reference Architectures
12. Integration with Other AWS Services
13. Best Practices Checklist

1. Overview of ECS

Amazon ECS is a fully managed container orchestration service designed for running Docker containers. It eliminates the need to install and manage your own container orchestration software or hardware.

2. Core ECS Components

Component	Description
Cluster	A logical grouping of ECS services and tasks.
Task Definition	Blueprint that defines how a container runs (image, CPU, memory, ports).
Task	An instantiation of a task definition.
Service	Manages long-running tasks, load balancing, and scaling.
Container Agent	Software that runs on EC2 instances and communicates with ECS.
Launch Types	Determines how ECS tasks are hosted (Fargate or EC2).

3. ECS Launch Types: EC2 vs Fargate

Feature	EC2 Launch Type	Fargate Launch Type
Infrastructure	User-managed EC2 instances	AWS-managed serverless compute
Networking	Shared ENI by default	Dedicated ENI per task
Scaling	EC2 Auto Scaling	Task-level auto scaling
Cost Model	Pay per instance	Pay per vCPU and memory usage per task
Use Cases	Custom AMIs, privileged containers	Simpler, isolated apps, microservices

4. ECS Networking Modes

- **Bridge**: Default Docker mode (EC2 only).
- **Host**: Shares host's network stack (EC2 only).
- **None**: No networking.
- **awsvpc**: Assigns an Elastic Network Interface (ENI) to the task — **recommended** for production.

 *Note*: Only **awsvpc** mode supports **Fargate**.

5. ECS Security Best Practices

- **IAM Roles for Tasks**: Fine-grained permissions per service.
 - **IAM Execution Role**: Used by ECS agent to pull images and send logs.
 - **Security Groups**: Attach at ENI level in **awsvpc** mode.
 - **Private Subnets**: Run services in private subnets with NAT Gateway.
 - **Secrets Management**: Store secrets in **AWS Secrets Manager** or **SSM Parameter Store**.
 - **Container Isolation**: Each Fargate task gets its own kernel.
-

6. Monitoring and Logging

Logs

- Use **CloudWatch Logs** with **awslogs** log driver.
- Optional: Forward logs to ELK, OpenSearch, or third-party services.

Metrics

- **CloudWatch Metrics:** CPU, memory usage.
- **Container Insights:** CPU, memory, network, storage metrics per container.

Tracing

- **AWS X-Ray:** Instrument services for tracing.
- ECS supports X-Ray daemon as a sidecar container.

7. CI/CD Pipeline with ECS

Typical CI/CD Flow:

1. Developer pushes code to **GitHub/CodeCommit**.
2. **CodeBuild** builds Docker image.
3. Image pushed to **ECR**.
4. ECS service is updated via **CodePipeline**.

Tools:

- **CodePipeline:** Orchestrate deployments.
- **CodeBuild:** Build Docker containers.
- **CodeDeploy:** Deploy ECS with Blue/Green support.

8. Scaling Strategies

ECS Service Auto Scaling

- **Target Tracking:** Scale based on CPU/memory thresholds.
- **Step Scaling:** Scale by fixed amount based on metric breach.

Task Scheduling

- Use scheduled tasks (cron) via **EventBridge**.

Spot Capacity

- Use **Fargate Spot** or **EC2 Spot Instances** for batch jobs and non-critical workloads.
-

9. Deployment Strategies

Rolling Updates (default)

- ECS replaces tasks in-place.

Blue/Green Deployments

- Use **CodeDeploy** with ECS.
 - Shift traffic using two **Target Groups** behind an ALB.
 - Supports linear, canary, and all-at-once rollout.
-

10. Cost Optimization

- Use **Fargate Spot** for dev/test workloads.
- Turn off idle ECS tasks or EC2 instances.
- Rightsize CPU/memory in task definitions.

- Leverage **Savings Plans** if using Fargate extensively.
 - Use **Compute Optimizer** recommendations.
-

11. Reference Architecture: Web App on ECS Fargate

Components:

- **Frontend:** React SPA on S3 + CloudFront
- **Backend:** ECS Fargate service with ALB
- **Database:** Amazon RDS
- **Secrets:** AWS Secrets Manager
- **Auth:** Amazon Cognito or JWT
- **Monitoring:** CloudWatch + X-Ray

Security:

- HTTPS via ALB
 - Private subnets + NAT Gateway
 - WAF for ALB
-

12. Integration with Other AWS Services

Service	Purpose
ECR	Container image storage
CloudWatch	Logs and metrics
App Mesh	Service mesh for inter-service communication

Cloud Map	DNS-based service discovery
CodeBuild/Deploy	CI/CD automation
Secrets Manager	Secure storage of environment secrets
X-Ray	Distributed tracing
S3	Static content or shared data store
EventBridge	Trigger ECS tasks on schedules or events

13. Best Practices Checklist

- Use **Fargate** unless you need custom EC2 configurations.
 - Use **awsvpc** networking mode for better isolation.
 - Use **IAM Roles for Tasks** with least privilege.
 - Store secrets in **Secrets Manager** or **SSM**.
 - Configure **logging and metrics** to CloudWatch.
 - Enable **auto scaling** with proper thresholds.
 - Use **Blue/Green deployments** for safer releases.
 - Tag ECS resources for cost tracking.
 - Use **Cloud Map** or service mesh for discovery.
 - Use **multi-AZ** deployment for high availability.
-